

Code review

The current implementation has a few lifecycle and state-management concerns. `GlobalScope` should not be used from a `ViewModel` because the coroutine isn't tied to the `ViewModel` lifecycle. Please use `viewModelScope`.

Also, `researchCalls` is exposed as a mutable list and updating it won't provide an observable state change to the UI. Please expose an immutable UI state through `StateFlow` (or another appropriate observable state mechanism).

Please also consider handling loading/error states and exceptions, and accessing the API through a repository rather than directly from the `ViewModel`. This will make the implementation lifecycle-safe, testable, and easier to maintain.

```
class ResearchViewModel {  
    var researchCalls = mutableListOf<String>()  
    fun fetchCalls() {  
        GlobalScope.launch {  
            val result = api.getCalls() // suspending function  
            researchCalls = result  
        }  
    }  
}
```

1. `GlobalScope.launch` should not be used

Avoid

`GlobalScope` in `ViewModel` code. The coroutine is not lifecycle-aware and can continue running after the `ViewModel` is cleared, potentially causing memory leaks, unnecessary network calls, or updating state that is no longer needed.

Use

```
viewModelScope.launch {  
}
```

This automatically cancels the coroutine when the ViewModel is cleared.

2. Assigning to var researchCalls

```
var researchCalls = mutableListOf<String>()
```

`mutableListOf` is exposed as mutable state

Any caller can modify it:

```
viewModel.researchCalls.add("something")
```

It will break encapsulation. We should prefer immutable state exposed to the UI.

3. Updating UI state from a coroutine needs a proper observable mechanism

```
researchCalls = result
```

doesn't notify the UI.

If Compose is observing this, simply changing a normal property won't trigger recomposition.

Use **StateFlow**:

```
_researchCalls.value = result
```

or expose it using **collectAsState()** in Compose.

4. No loading state

The UI has no way to know whether the API request is:

- Not started
- Loading
- Successful
- Failed

For example:

```
sealed interface ResearchUiState {  
    data object Loading : ResearchUiState  
    data class Success(val calls: List<String>) : ResearchUiState  
    data class Error(val message: String) : ResearchUiState  
}
```

5. **ViewModel** should extend **ViewModel**

The class should be:

```
class ResearchViewModel : ViewModel()
```

Otherwise, you don't get access to Android's lifecycle-aware **viewModelScope**.

6. No exception handling

If `api.getCalls()` throws an exception, the coroutine fails.

We should use

```
viewModelScope.launch {  
  
    try {  
  
        val result = api.getCalls()  
  
        _researchCalls.value = result  
  
    } catch (e: Exception) {  
  
        // update error state  
  
    }  
  
}
```